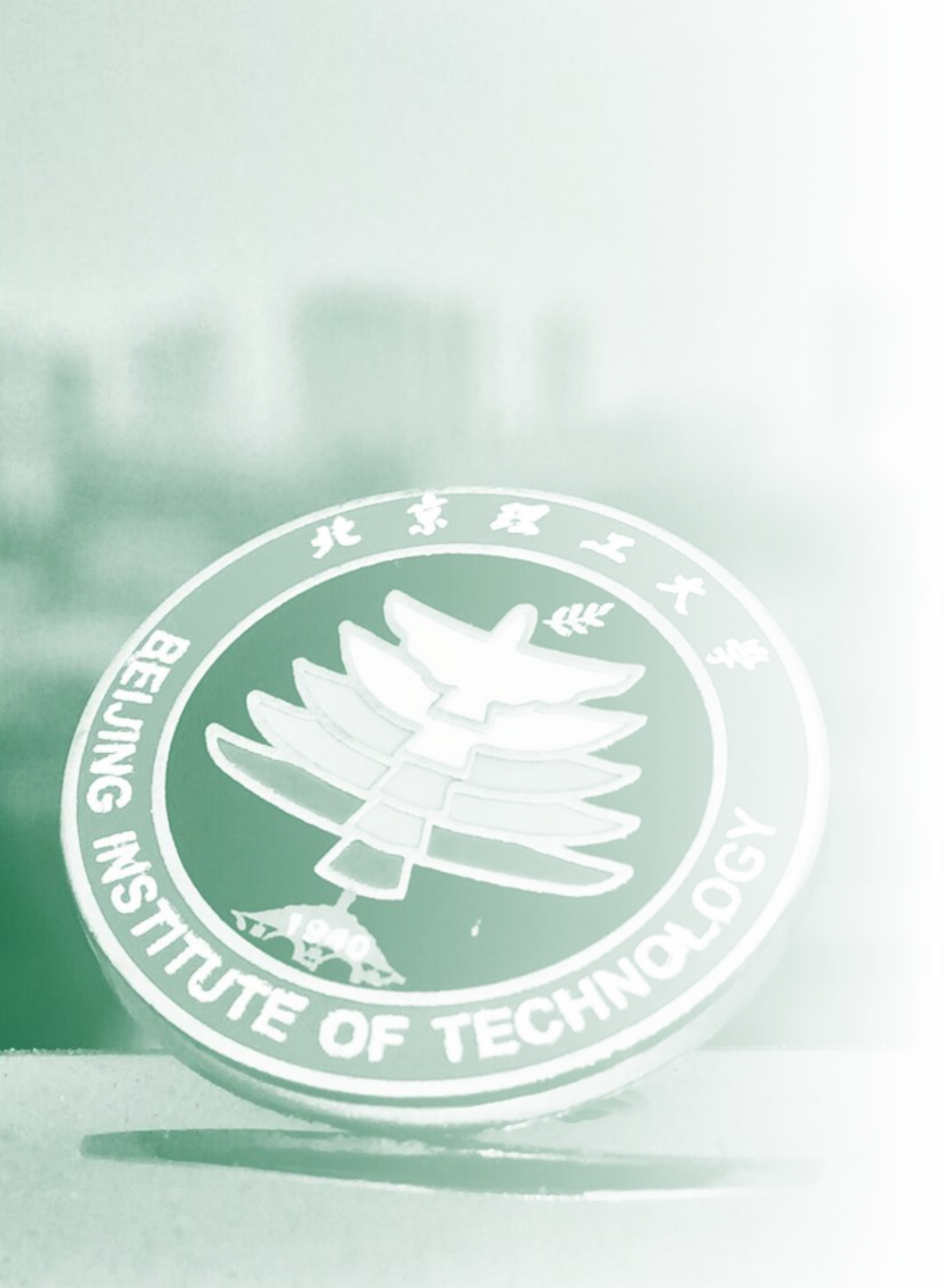
◁ BIT ▷

高斯数据库技术实验指导

时间：2024/4/17

北京理工大學
BEIJING INSTITUTE OF TECHNOLOGY

德以明理　学以精工



目录 | CONTENTS

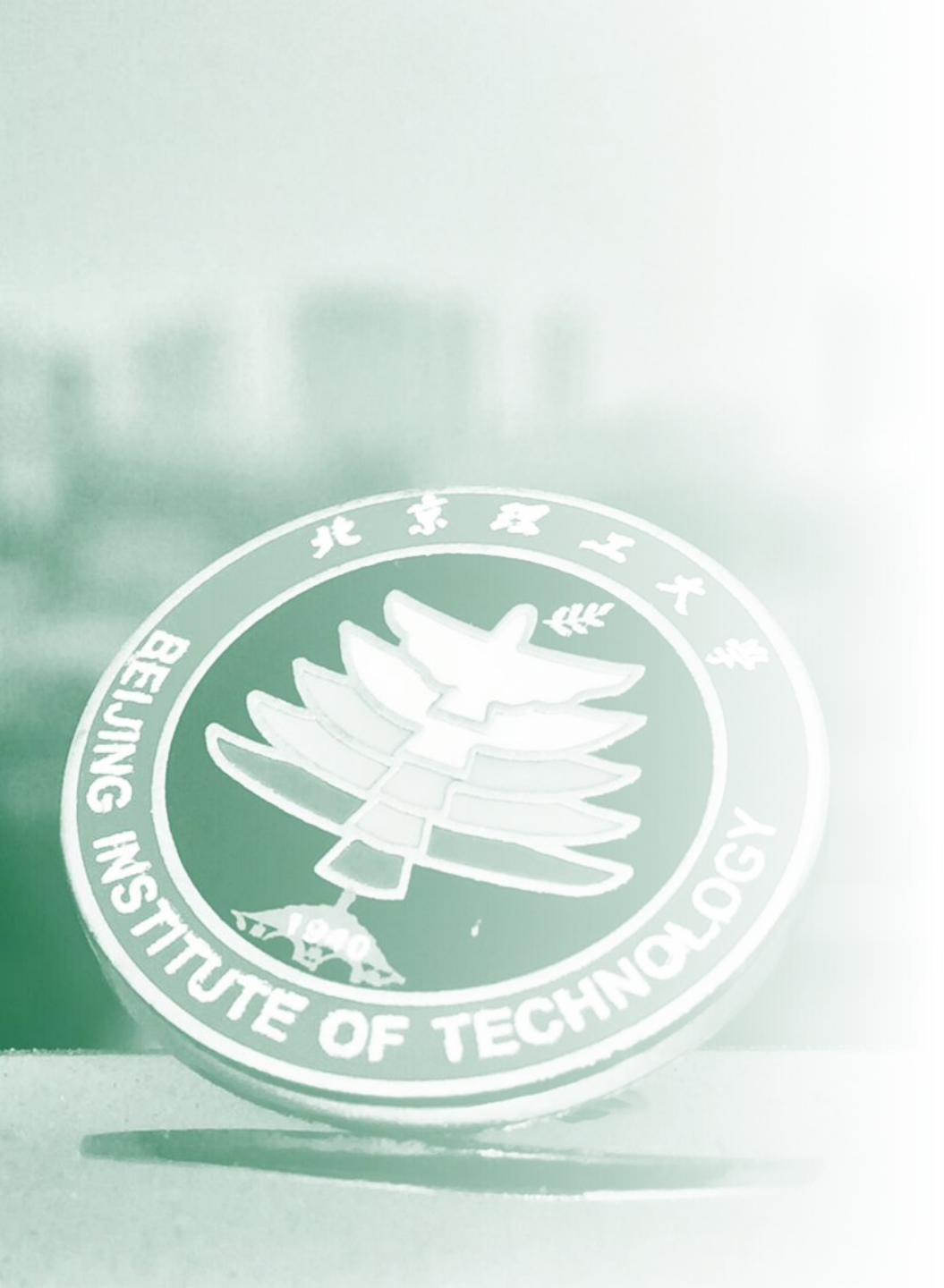
1 购买弹性云服务器ECS

2 安装openGauss数据库

3 使用工具连接ECS

4 使用简单的SQL语句

5 创建并使用IAM帐号



1 购买弹性云服务器ECS

购买弹性云服务器ECS

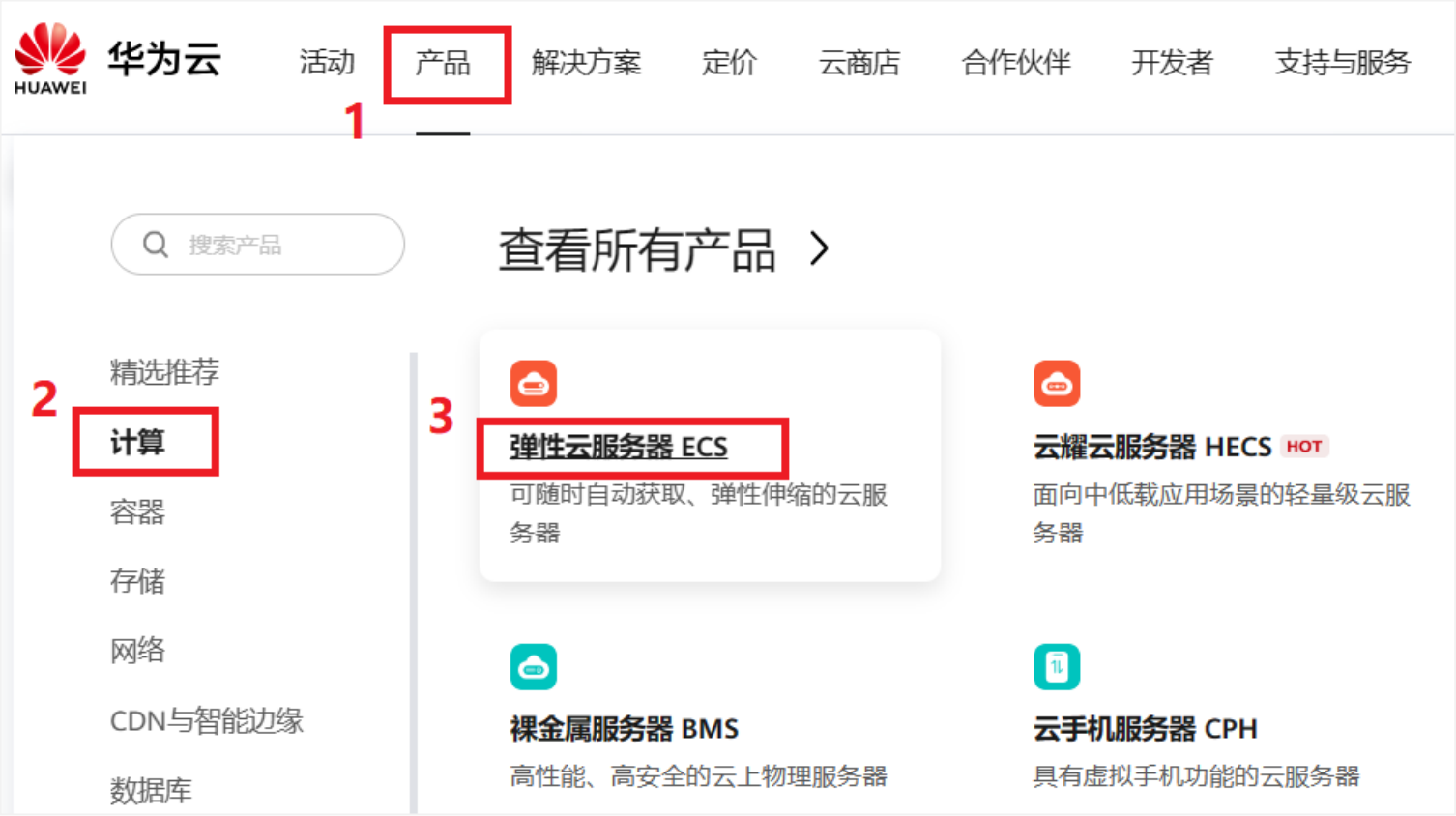
进入华为云官网 https://www.huaweicloud.com/ ，单击登录。

购买弹性云服务器ECS

输入账号名和密码，单击登录。
如果还没有注册，单击注册，按步骤注册后进行登录。

购买弹性云服务器ECS

在华为云主页（https://www.huaweicloud.com/）点击产品，选择＂计算＂，再选择＂弹性云服务器ECS＂。



购买弹性云服务器ECS

进入弹性云服务器ECS购买界面。

购买弹性云服务器ECS

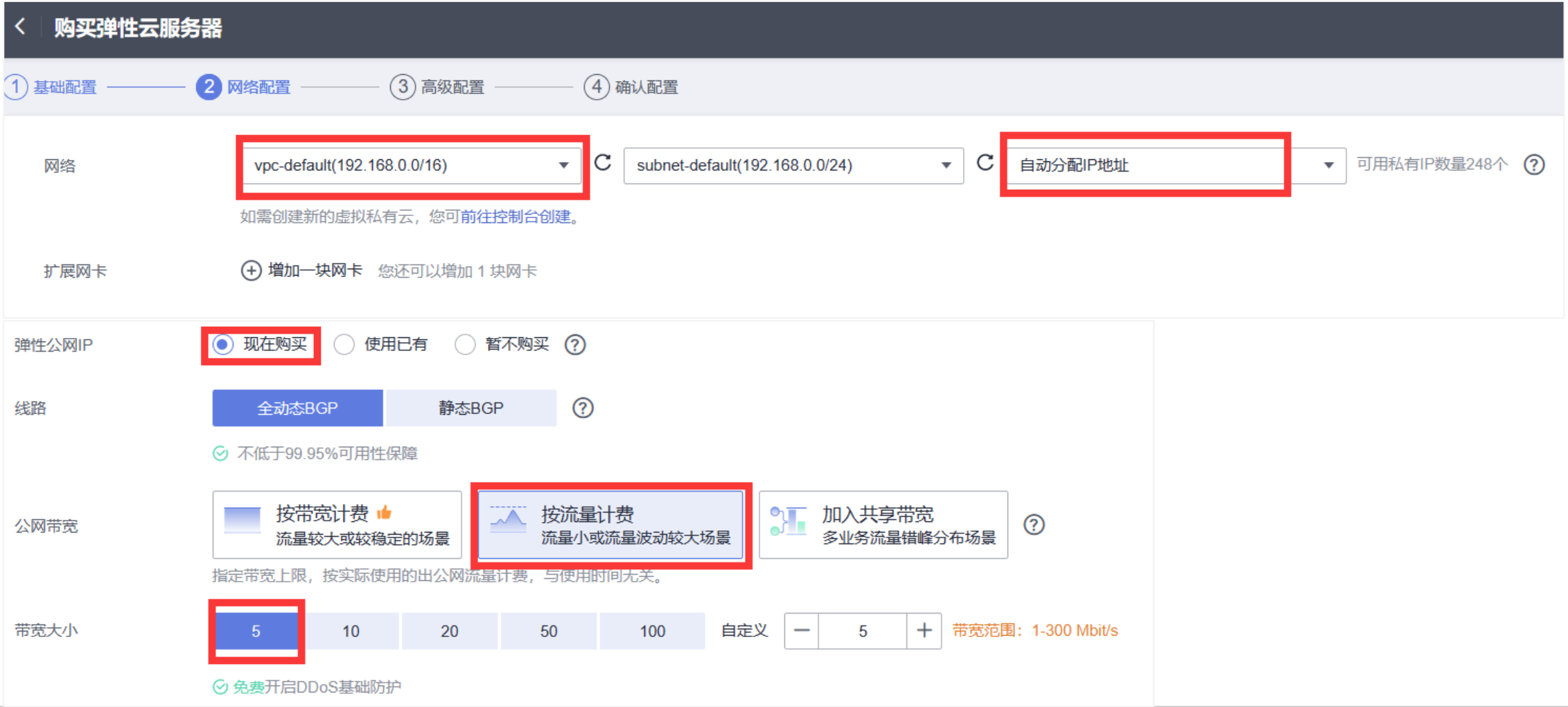
进行ECS基础配置。

购买弹性云服务器ECS

进行ECS基础配置。完成后，单击“下一步：网络配置”。

购买弹性云服务器ECS

进行ECS网络配置。安全组选择“default”。完成后，单击“下一步：高级配置”。



购买弹性云服务器ECS

记住用户名为root，输入自定义密码和确认密码，其余默认，单击“下一步：确认配置 "。

购买弹性云服务器ECS

确认设置信息，尤其是配置费用，然后勾选协议，单击“立即购买”。

配置

基础配置

计费模式	按需计费	区域	北京四
规格	鲲鹏通用计算增强型 | kc1.large.2 | 2vCPUs | 4...	镜像	openEuler 20.03 64bit with ARM
系统盘	通用型SSD, 40GiB		

网络配置

虚拟私有云	vpc-default(192.168.0.0/16)	安全组	Sys-WebServer
弹性公网IP	全动态BGP | 计费方式: 按流量计费 | 带宽: 5 M...		

高级配置

云服务器名称	ecs-7cf0	登录凭证	密码

生成Open API最佳实践脚本

购买数量 — 1 + 您最多可以创建198台云服务器。 申请更多云服务器配额请单击申请扩大配额。

协议 我已经阅读并同意《镜像免责声明》

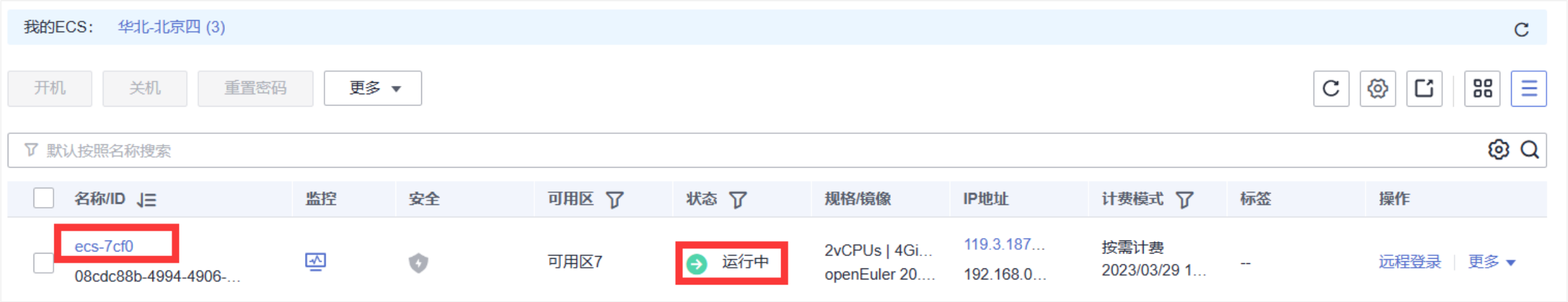
任务提交成功！

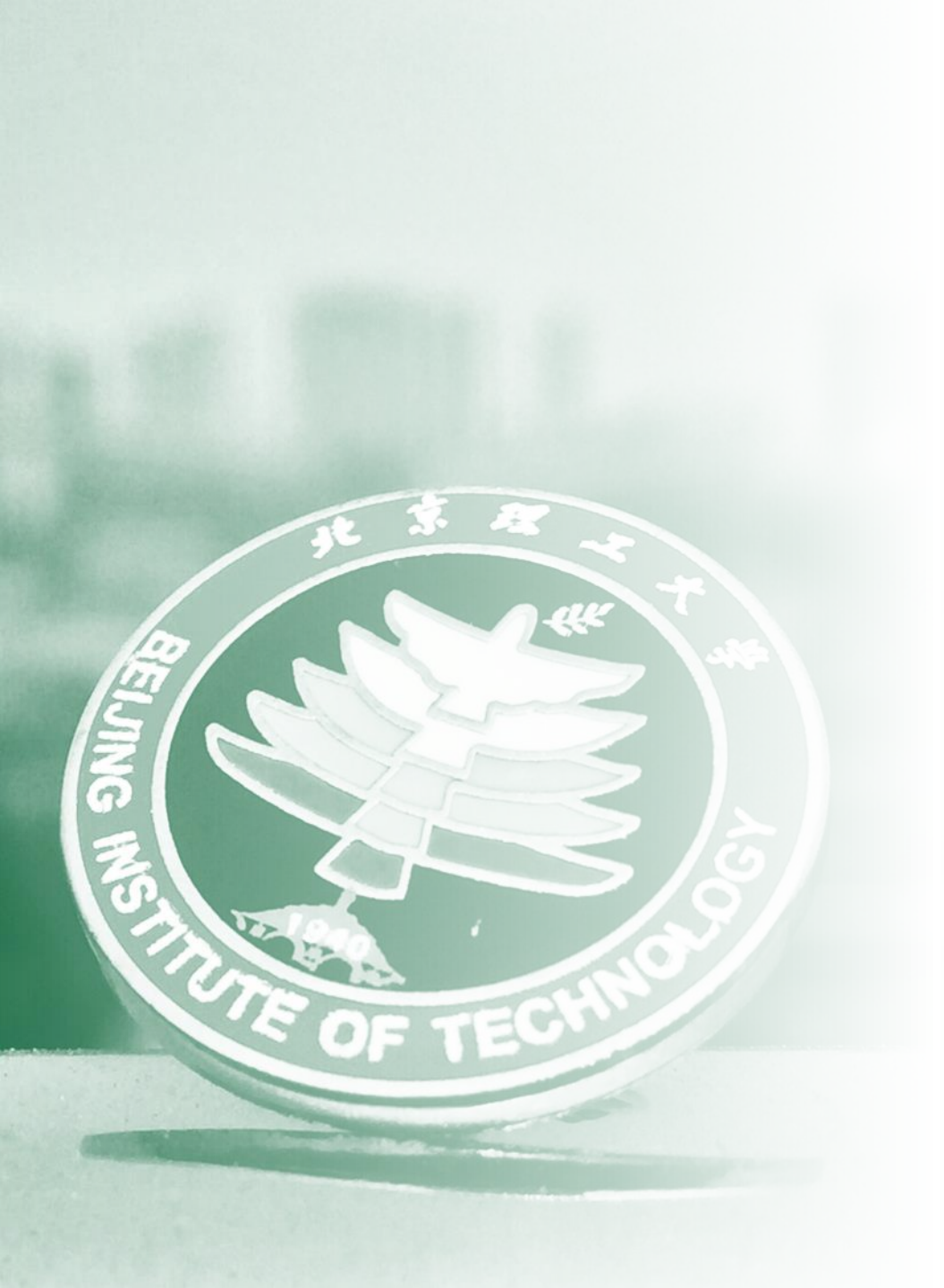
您的弹性云服务器ecs-7cf0已经开始创建。

返回购买 返回云服务器列表

购买弹性云服务器ECS

查看云服务器列表。状态列显示“运行中”，则表示购买成功。





2 安装openGauss数据库

使用“CloudShell登录”连接ECS

对购买好的ECS服务器，点击“更多”、“开机”，开机之后，点击“远程登录”。

使用“CloudShell登录”连接ECS

点击“远程登录”之后，点击“CloudShell登录”。

登录Linux弹性云服务器

使用CloudShell登录New!　　登录不上?

请确保安全组已放通CloudShell连接实例使用的端口（默认使用22端口）

优势：操作更流畅，命令支持复制粘贴，支持浏览输出历史和多终端分区布局。了解更多

CloudShell登录

其他方式

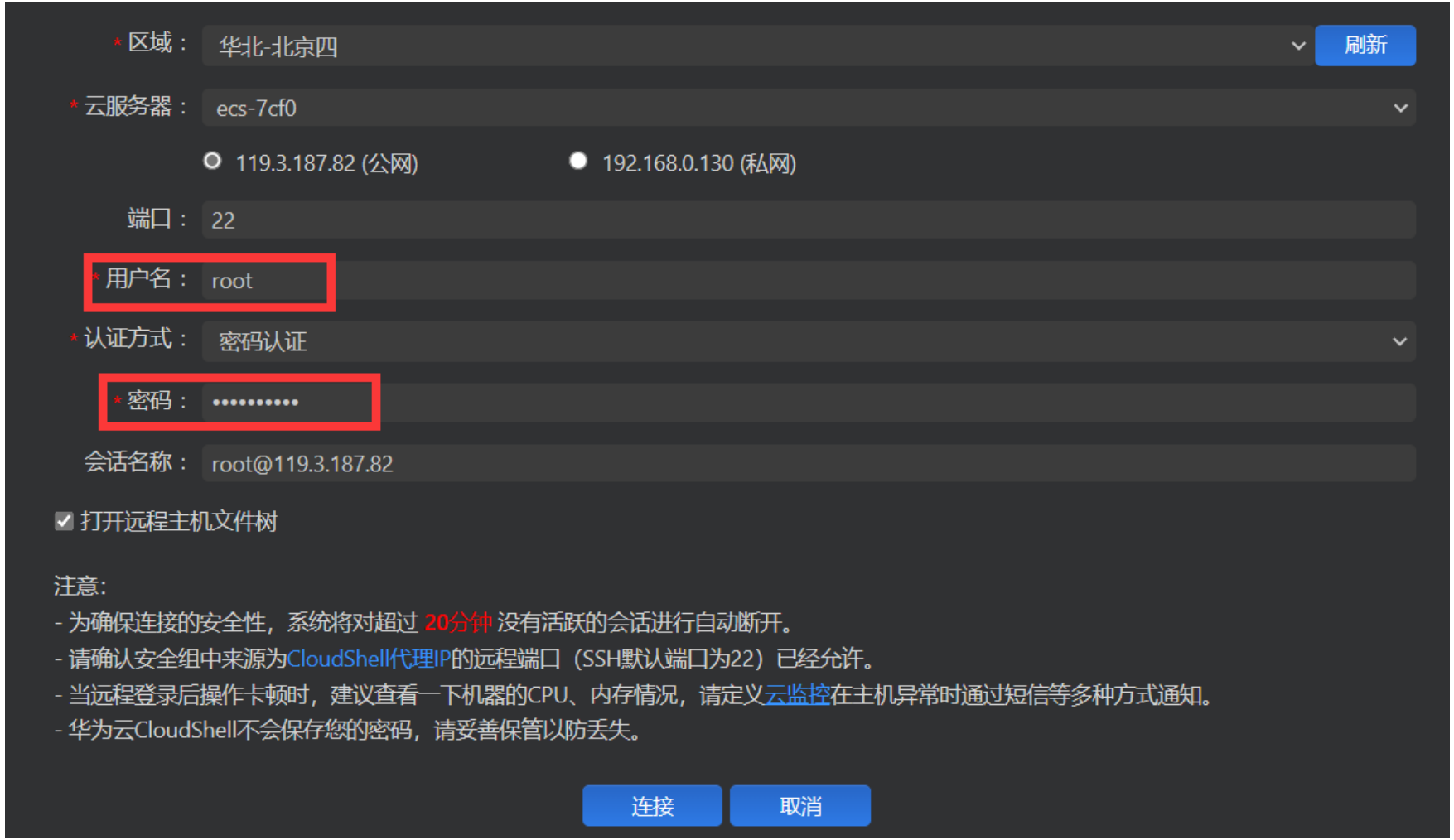
1、使用控制台提供的VNC方式登录

立即登录

使用“CloudShell登录”连接ECS

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

等加载完成后，输入root用户密码后点击＂连接＂按钮即可登录。



修改操作系统配置

设置字符集参数： 将各数据库节点的字符集设置为相同的字符集。

```
[root@ecs-58ab ~]# cat>>/etc/profile<<EOF
>
> export LANG=en_US.UTF-8
> EOF
[root@ecs-58ab ~]# source /etc/profile


Welcome to 4.19.90-2003.4.0.0036.oe1.aarch64
```

```
# cat>>/etc/profile<<EOF
>export LANG=en_US.UTF-8
>EOF
# source /etc/profile
```

修改操作系统配置

修改python版本并安装libaio包：

1.进入/usr/bin目录
2.备份python文件
3.建立Python3软连接
4.验证Python版本
5.下载安装libaio包

```
[root@ecs-58ab ~]# cd /usr/bin
[root@ecs-58ab bin]# mv python python.bak
[root@ecs-58ab bin]# ln -s python3 /usr/bin/python
[root@ecs-58ab bin]# python -V
Python 3.7.4
[root@ecs-58ab bin]# yum install libaio* -y
```

```
# cd /usr/bin
# mv python python.bak
# ln -s python3 /usr/bin/python
# python -V
```

(这里应该输出3.7.x)

```
# yum install libaio* -y
```

下载数据库安装包

以root用户登录待安装openGauss的主机，并创建存放安装包的目录。

```
[root@ecs-58ab bin]# mkdir -p /opt/software/openGauss
```

openGauss安装用户omm须具有/opt/software/openGauss目录的读写权限。

```
[root@ecs-58ab bin]# chmod 755 -R /opt/software
```

切换到安装目录。

```
[root@ecs-58ab bin]# cd /opt/software/openGauss
```

使用wget下载数据库安装包到安装目录：

```
[root@ecs-58ab openGauss]# wget https://opengauss.obs.cn-south-
1.myhuaweicloud.com/2.0.0/arm/openGauss-2.0.0-openEuler-64bit-a
ll.tar.gz
```

mkdir -p /opt/software/openGauss
chmod 755 -R /opt/software
cd /opt/software/openGauss
wget https://opengauss.obs.cn-south-1.myhuaweicloud.com/2.0.0/arm/openGauss-2.0.0-openEuler-64bit-all.tar.gz

创建XML配置文件

XML文件包含部署openGauss的服务器信息、安装路径、IP地址以及端口号等，用于告知openGauss如何部署。

这里以单节点配置的方案为例，说明如何创建XML配置文件。

以root用户登录待安装openGauss的主机，切换到存放安装包的目录。
创建XML配置文件，用于数据库安装。

```
[root@ecs-58ab openGauss]# cd /opt/software/openGauss
[root@ecs-58ab openGauss]# vi clusterconfig.xml
```

cd /opt/software/openGauss
vi clusterconfig.xml

创建XML配置文件

输入“i”进入INSERT模式，添加文本（共29行）如下。
之后点击“Esc”退出INSERT模式，然后输入“:wq!”后回车。

这里的nodeNames、backIp1s改为自己的ECS服务器的主机名和私有IP地址。

```
1.<?xml version="1.0" encoding="UTF-8"?>
2.<ROOT>
3.    <CLUSTER>
4.        <PARAM name="clusterName" value="dbCluster" />
5.        <PARAM name="nodeNames" value="ecs-7cf0" />
6.        <PARAM name="backIp1s" value="192.168.0.130"/>
7.        <PARAM name="gaussdbAppPath" value="/opt/gaussdb/app" />
8.        <PARAM name="gaussdbLogPath" value="/var/log/gaussdb" />
9.        <PARAM name="gaussdbToolPath" value="/opt/huawei/wisequery" />
10.        <PARAM name="corePath" value="/opt/opengauss/corefile"/>
11.        <PARAM name="clusterType" value="single-inst"/>
12.    </CLUSTER>
13.
```

创建XML配置文件


```
14.     <DEVICELIST>
15.
16.         <DEVICE sn="1000001">
17.             <PARAM name="name" value="ecs-7cf0"/>
18.             <PARAM name="azName" value="AZ1"/>
19.             <PARAM name="azPriority" value="1"/>
20.             <PARAM name="backIp1" value="192.168.0.130"/>
21.             <PARAM name="sshIp1" value="192.168.0.130"/>
22.
23.      <!--dbnode-->
24.      <PARAM name="dataNum" value="1"/>
25.      <PARAM name="dataPortBase" value="26000"/>
26.      <PARAM name="dataNode1" value="/gaussdb/data/db1"/>
27.         </DEVICE>
28.     </DEVICELIST>
29.</ROOT>
```

创建XML配置文件


```xml
<?xml version="1.0" encoding="UTF-8"?>
<ROOT>
  <CLUSTER>
    <PARAM name="clusterName" value="dbCluster" />
    <PARAM name="nodeNames" value="ecs-a4d3" />
    <PARAM name="backIp1s" value="192.168.0.58"/>
    <PARAM name="gaussdbAppPath" value="/opt/gaussdb/app" />
    <PARAM name="gaussdbLogPath" value="/var/log/gaussdb" />
    <PARAM name="gaussdbToolPath" value="/opt/huawei/wisequery" />
    <PARAM name="corePath" value="/opt/opengauss/corefile"/>
    <PARAM name="clusterType" value="single-inst"/>
  </CLUSTER>

  <DEVICELIST>
    <DEVICE sn="1000001">
      <PARAM name="name" value="ecs-a4d3"/>
      <PARAM name="azName" value="AZ1"/>
      <PARAM name="azPriority" value="1"/>
      <PARAM name="backIp1" value="192.168.0.58"/>
      <PARAM name="sshIp1" value="192.168.0.58"/>

        <!--dbnode-->
        <PARAM name="dataNum" value="1"/>
        <PARAM name="dataPortBase" value="26000"/>
        <PARAM name="dataNode1" value="/gaussdb/data/db1"/>
    </DEVICE>
  </DEVICELIST>
</ROOT>
```

初始化安装环境

使用vi打开文件“/etc/profile.d/performance.sh”。

```
[root@ecs-58ab openGauss]# vi /etc/profile.d/performance.sh
```

输入“i”，进入INSERT模式。
用“#”注释掉“sysctl -w vm.min_free_kbytes =112 640 &> /dev/null” 这行。

```
CPUNO=`cat /proc/cpuinfo|grep processor|wc -l`
export GOMP_CPU_AFFINITY=0-$[CPUNO - 1]

#sysctl -w vm.min_free_kbytes=112640 &> /dev/null
sysctl -w vm.dirty_ratio=60 &> /dev/null
sysctl -w kernel.sched_autogroup_enabled=0 &> /dev/null
~
~
```

① # vi /etc/profile.d/performance.sh
② 用“#”注释掉“sysctl -w vm.min_free_kbytes =112 640 &> /dev/null” 这行

初始化安装环境

为确保openssl版本正确，执行预安装前加载安装包中lib库。需要执行以下命令：

① 使用vi打开文件/etc/profile。

```
[root@ecs-58ab openGauss]# vi /etc/profile
```

② 在文件底部添加以下内容。

export packagePath=/opt/software/openGauss
export LD_LIBRARY_PATH=$packagePath/script/gspylib/clib:$LD_LIBRARY_ PATH

③ 配置完成后，使用命令 source /etc/profile 使设置生效。

vi /etc/profile
export packagePath=/opt/software/openGauss
export LD_LIBRARY_PATH=$packagePath/script/gspylib/clib:$LD_LIBRARY_ PATH
source /etc/profile

初始化安装环境

在安装目录下，按顺序解压以下两个安装包
openGauss-2.0.0-openEuler-64bit-all.tar.gz
openGauss-2.0.0-openEuler-64bit-om.tar.gz

```
[root@ecs-58ab openGauss]#  tar -zxvf  openGauss-2.0.0-openEuler-64bit-om.tar.gz
```

```
[root@ecs-58ab openGauss]# tar -zxvf  openGauss-2.0.0-openEuler-64bit-all.tar.gz
```

tar -zxvf openGauss-2.0.0-openEuler-64bit-all.tar.gz
tar -zxvf openGauss-2.0.0-openEuler-64bit-om.tar.gz

初始化安装环境

安装包解压后，会在/opt/software/openGauss路径下自动生成script子目录，并且在script目录下生成gs_preinstall等各种OM工具脚本。

```
[root@ecs-58ab script]# cd /opt/software/openGauss/script/
[root@ecs-58ab script]# ls
config      gs_checkperf  gs_install        gspylib      gs_upgradectl  local
gs_backup   gs_collector  gs_om             gs_ssh       impl           transfer.py
gs_check    gs_dropnode   gs_postuninstall  gs_sshexkey  __init__.py
gs_checkos  gs_expansion  gs_preinstall     gs_uninstall killall
```

```
# cd /opt/software/openGauss/script/
# ls
```

初始化安装环境

切换到script目录，输入以下命令来创建openGauss omm用户互信：

python gs_preinstall -U omm -G dbgrp -X /opt/software/openGauss/clusterconfig.xml

命令中的omm为操作系统用户，
dbgrp为运行openGauss的操作系统用户的群组名称，
/opt/software/openGauss/clusterconfig.xml为openGauss配置文件路径。

 # python gs_preinstall -U omm -G dbgrp -X /opt/software/openGauss/clusterconfig.xml

初始化安装环境

这里提示你是否创建互信。填写 yes。

```
[root@ecs-5451 script]# python gs_preinstall -U omm -G dbgrp -X /opt/software/openGauss/clusterconfig.xml
Parsing the configuration file.
Successfully parsed the configuration file.
Installing the tools on the local node.
Successfully installed the tools on the local node.
Setting pssh path
Successfully set core path.
Are you sure you want to create the user[omm] and create trust for it (yes/no)? yes
```

这里要求你输入两次密码。

```
Please enter password for cluster user.
Password:
Please enter password for cluster user again.
Password:
```

```
Successfully set ARM Optimization.
Fixing server package owner.
Setting finish flag.
Successfully set finish flag.
Preinstallation succeeded.
[root@ecs-5451 script]#
```

初始化安装环境

修改/opt/software/openGauss/script文件的权限，登录到openGauss的主机，并切换到omm用户。

```
[root@ecs-5451 script]# chmod -R 755 /opt/software/openGauss/script
[root@ecs-5451 script]# su - omm
Last login: Wed Apr 17 01:16:33 CST 2024 on pts/0


Welcome to 4.19.90-2110.8.0.0119.oe1.aarch64

System information as of time:  Wed Apr 17 01:23:20 CST 2024

System load:    0.00
```

chmod -R 755 /opt/software/openGauss/script
su - omm

执行安装openGauss

使用gs_install命令安装openGauss：

中间需要输入两次密码，成功后如下图。

```
Configuring pg_hba on all nodes.
Configuration is completed.
Successfully started cluster.
Successfully installed application.
end deploy..
```

$ gs_install -X /opt/software/openGauss/clusterconfig.xml --gsinit-parameter="--encoding=UTF8" --dn-guc="max_process_memory=4GB" --dn-guc="shared_buffers=256MB" --dn-guc="bulk_write_ring_size=256MB" --dn-guc="cstore_buffers=16MB"

（整个粘上去）

执行安装openGauss

openGauss安装成功

openGauss数据库的简单使用

在数据库主节点服务器上，在omm操作系统用户环境下：
使用 **gs_om -t status** 命令查看服务是否启动。

```
[omm@ecs-58ab ~]$ gs_om -t status
-----------------------------------------------------------------------

cluster_name     : dbCluster
cluster_state    : Normal
redistributing   : No

-----------------------------------------------------------------------
```

如果没有启动，使用 **gs_om -t start** 命令启动数据库服务。

```
[omm@ecs-58ab ~]$ gs_om -t start
Starting cluster.
=========================================
[SUCCESS] ecs-58ab:
[2022-10-05 11:47:33.823][15563][][gs_ctl]: gs_ctl started,datadir is /gaussdb/data/d
[2022-10-05 11:47:33.826][15563][][gs_ctl]:  another server might be running; Please
rt command
=========================================
Successfully started.
```

$ gs_om -t status
$ gs_om -t start（如果没有启动，才执行这个命令）

openGauss数据库的简单使用

使用 **gsql -d postgres -p 26000 -r** 命令连接数据库。

```
[omm@ecs-58ab ~]$ gsql -d postgres -p 26000 -r
gsql ((openGauss 2.0.0 build 78689da9) compiled at 2021-03-31 21:03:52 commit 0 last
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

postgres=#
```

出现上图即连接数据库成功。

命令中的postgres是openGauss安装完成后默认生成的数据库，初始可以连接到此数据库进行新数据库的创建。

26000为数据库主节点的端口号，需根据openGauss的实际情况做替换。

$ gsql -d postgres -p 26000 -r

openGauss数据库的简单使用

默认只有openGauss安装时创建的管理员用户（这里是omm）可以访问初始数据库。还可以使用 **CREATE USER username WITH PASSWORD "password";** 命令创建其他数据库用户帐号。下图成功创建了用户john。

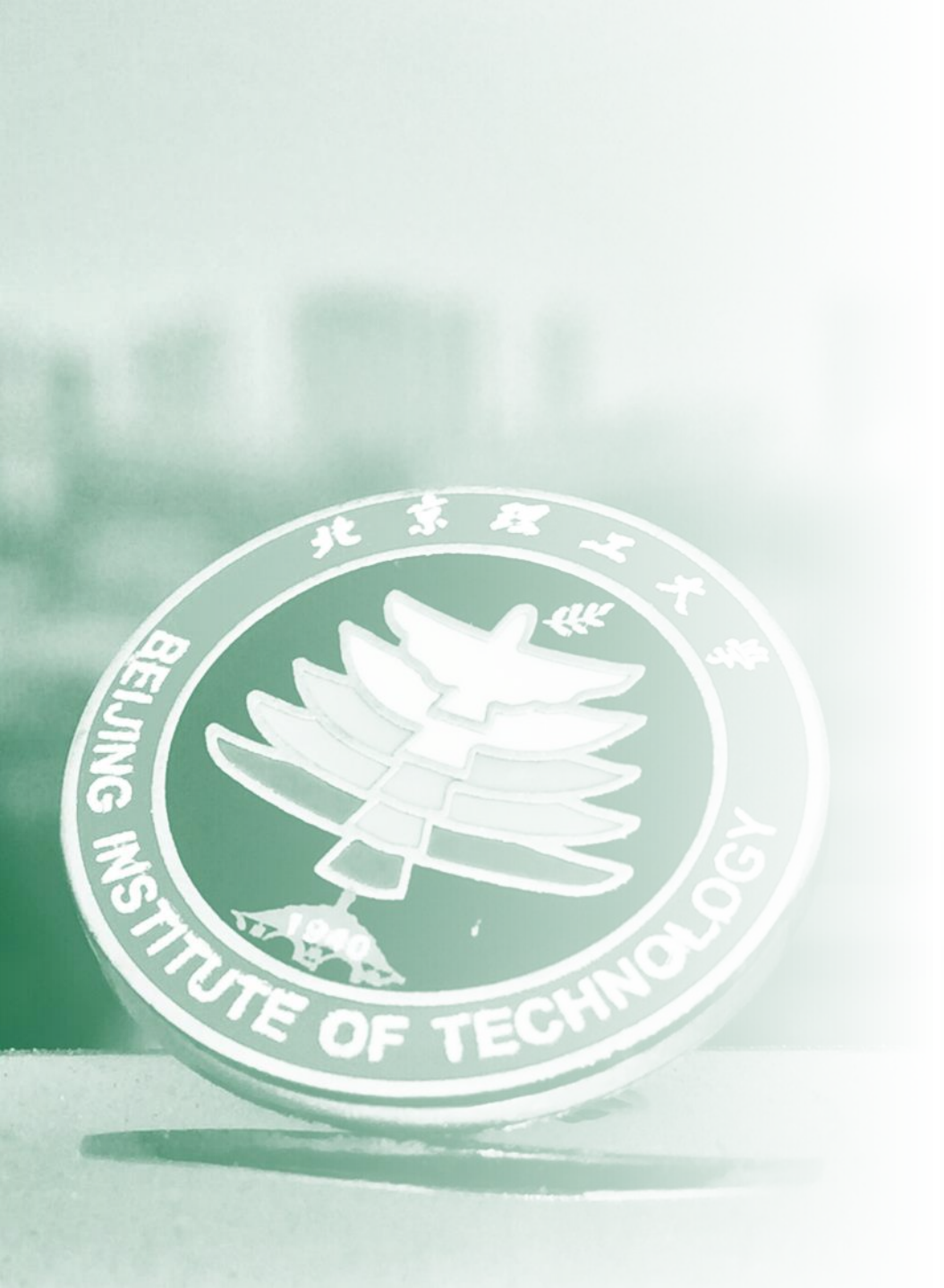
```
postgres=# CREATE USER john WITH PASSWORD "399ng259dsn..";
CREATE ROLE
```

可以使用命令 **CREATE DATABASE database_name OWNER owner_name;** 创建数据库。下图成功创建了数据库newdb，管理员设为john。

```
postgres=# CREATE DATABASE newdb OWNER john;
CREATE DATABASE
```

Ctrl+Z即可退出数据库。

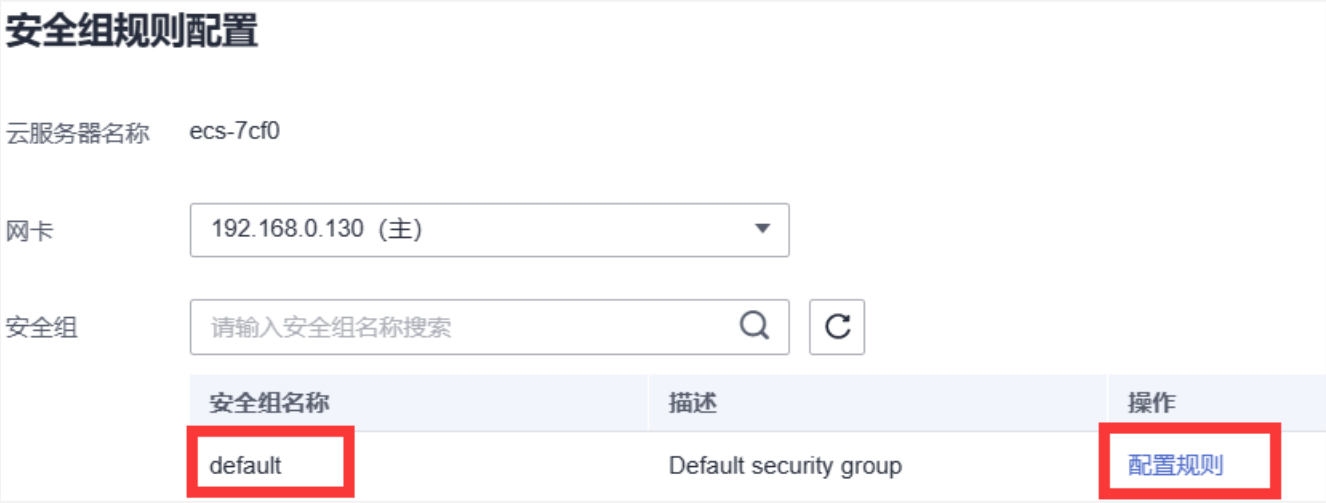
postgres=# CREATE USER john WITH PASSWORD "399ng259dsn..";
postgres=# CREATE DATABASE newdb OWNER john;



3 使用工具连接ECS

使用Data Studio连接ECS

打开ECS云服务器列表，点击“更多” “网络设置” “安全组规则配置”。
下图的云服务器的安全组是default，点击“配置规则”。



使用Data Studio连接ECS

点击“入方向规则” “添加规则” 。

输入“优先级”为1，“协议端口”为26000，作为后面连接数据库的端口。

使用Data Studio连接ECS

初始情况下，在配置文件pg_hba.conf中规定了不允许远程登录。所以需要修改该文件允许远程登录。

最后一行命令的意思是：对于主机配置（host），允许所有远程主机（0/0），以所有用户身份（all），以密码sha256加密的方式（sha256），登录所有的库（all）。

```
Total instances: 1. Failed instances: 0.
ALL: Success to perform gs_guc!
```

[omm@db1 ~]$ **cd /gaussdb/data/db1**
[omm@db1 db1]$ **cp pg_hba.conf pg_hba.conf.bak**
[omm@db1 db1]$ **gs_guc set -N all -I all -h "host all all 0.0.0.0/0 sha256"**

使用Data Studio连接ECS

登录数据库并创建＂dboper＂用户，密码为＂**dboper@123**＂（密码可自定义），同时进行授权，并退出数据库。

```
[omm@ecs-35b6 ~]$ gsql -d postgres -p 26000 -r
gsql ((openGauss 2.0.0 build 7b089da9) compiled at 2021-03-31 21:03:52 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

postgres=# CREATE USER dboper IDENTIFIED BY 'dboper@123';
CREATE ROLE
postgres=# alter user dboper sysadmin;
ALTER ROLE
postgres=# \q
```

[omm@db1 db1]$ **gsql -d postgres -p 26000 -r**

postgres=#**CREATE USER dboper IDENTIFIED BY 'dboper@123';**

CREATE ROLE

postgres=#**alter user dboper sysadmin;**

ALTER ROLE

postgres=# **\q**

使用Data Studio连接ECS

修改数据库监听地址。
主要是将 listen_addresses 的值由原来 IP 地址修改成为*号。
修改完成后重启数据库生效。

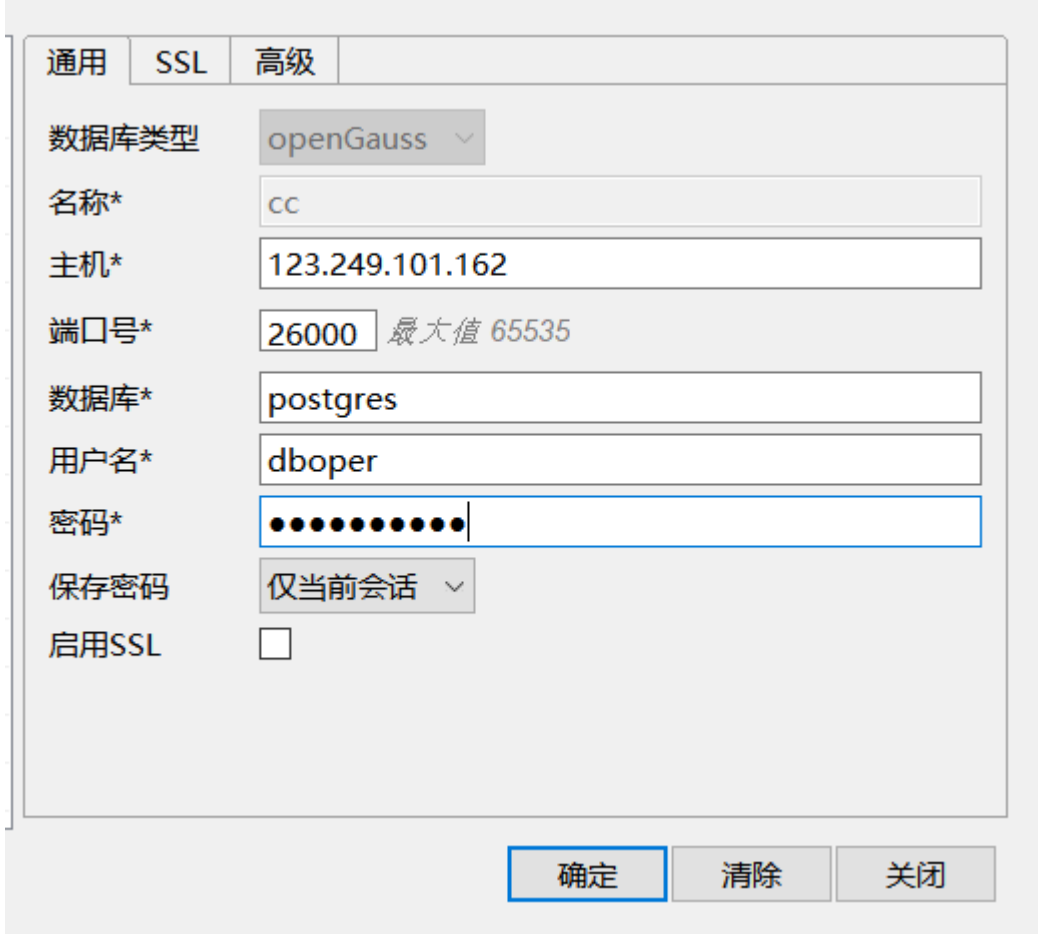
```
[omm@ecs-35b6 ~]$ cd /gaussdb/data/db1
[omm@ecs-35b6 db1]$ cp postgresql.conf postgresql.conf.bak
[omm@ecs-35b6 db1]$ gs_guc set -I all -c "listen_addresses='*'"
expected instance path: [/gaussdb/data/db1/postgresql.conf]
gs_guc set: listen_addresses='*': [/gaussdb/data/db1/postgresql.conf]

Total instances: 1. Failed instances: 0.
Success to perform gs_guc!
```

[omm@db1 db1]$ **cd /gaussdb/data/db1**
[omm@db1 db1]$ **cp postgresql.conf postgresql.conf.bak**
[omm@db1 db1]$ **gs_guc set -I all -c "listen_addresses='*'"**
[omm@db1 db1]$ **gs_om -t restart**

使用Data Studio连接ECS

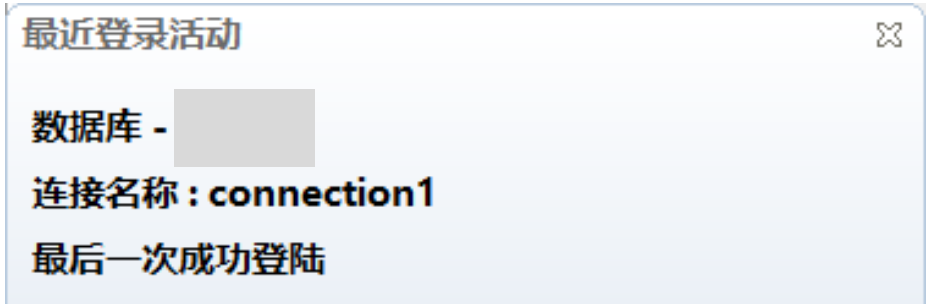
在本地电脑打开Data Studio，点击“文件” “新建连接”。

比如连接数据库newdb，如右图：
其中
名称：随意输入
主机：ECS的公网IP地址
端口号：26000
数据库：postgres
用户名：dboper
密码：dboper@123
点击确定建立数据库连接。



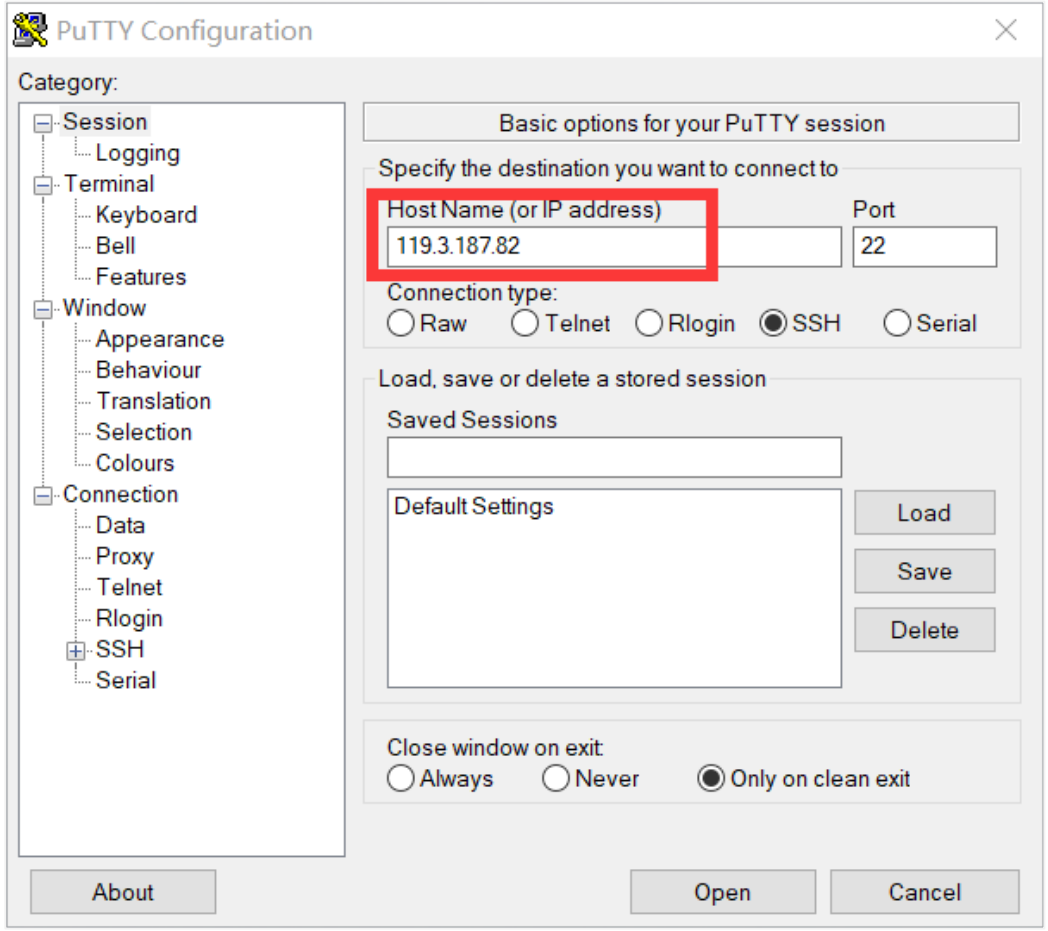
使用Data Studio连接ECS

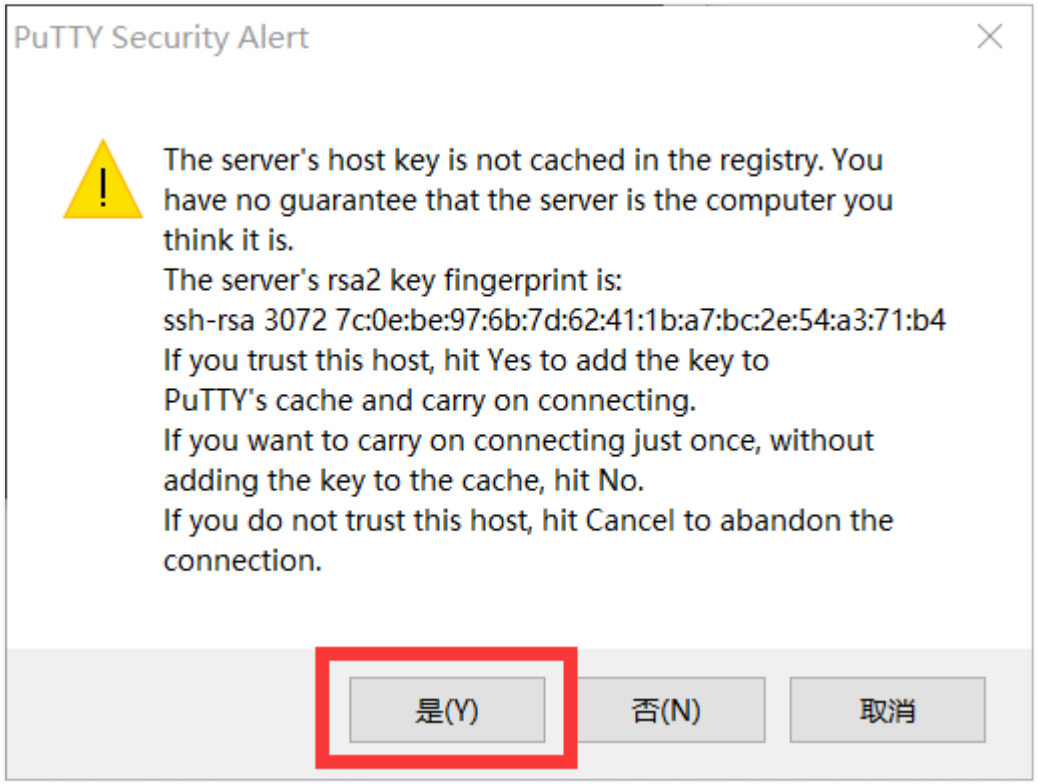
弹窗提示表示连接成功。



使用putty连接ECS

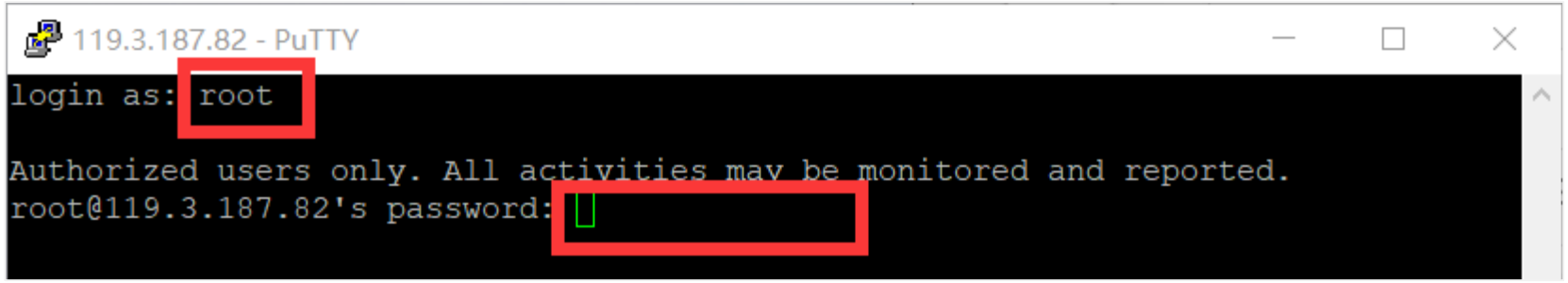
在本地电脑打开putty，在Host Name的地方输入购买的ECS服务器的公网IP地址，点击“Open”。如果弹出右图的窗口，点击“是”。



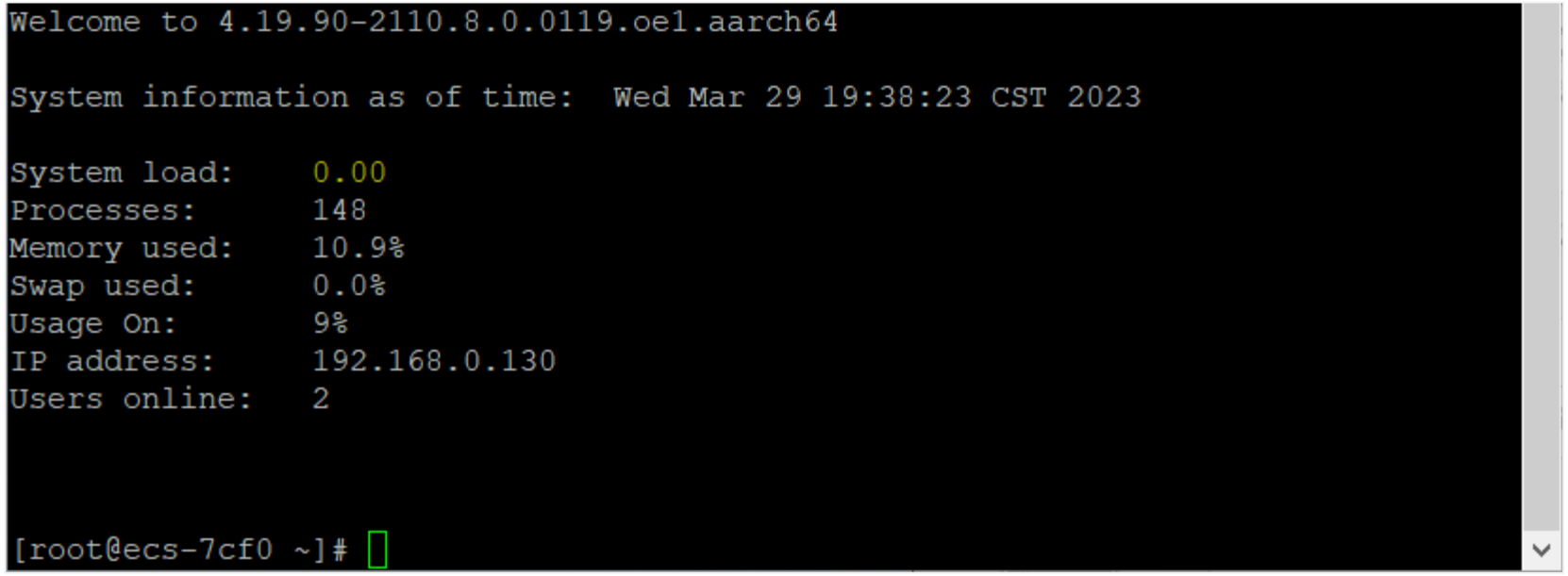


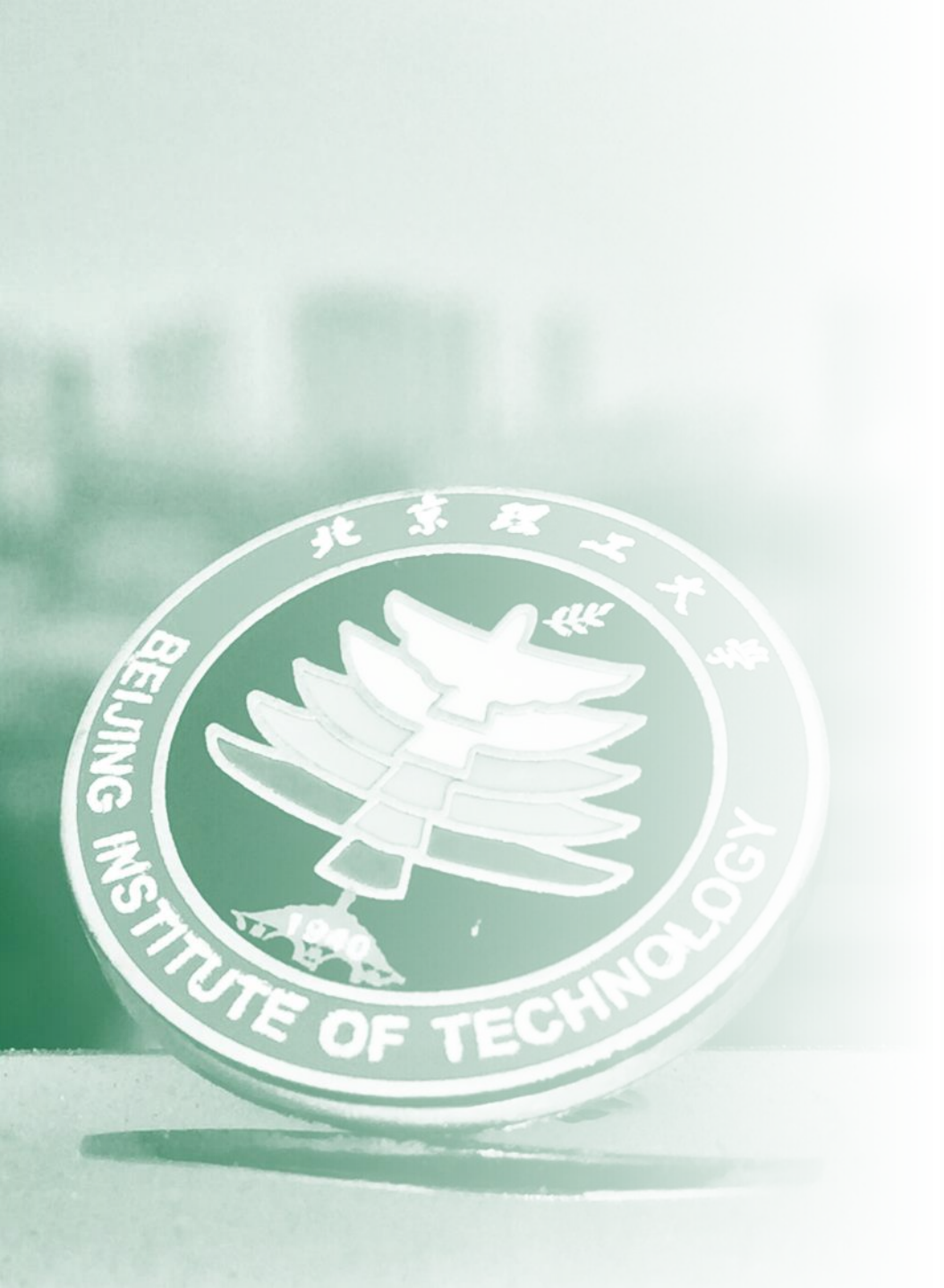
使用putty连接ECS

以root用户来登录。输入密码。



提示如下，即连接成功。





4 使用简单的SQL语句

使用简单的SQL语句

创建表空间fastspace。

CREATE TABLESPACE fastspace RELATIVE LOCATION 'tablespace/tablespace_1';

```
postgres=# CREATE TABLESPACE fastspace RELATIVE LOCATION 'tablespace/tablespace_1';
CREATE TABLESPACE
```

将“fastspace”表空间的访问权限赋予数据用户john。

GRANT CREATE ON TABLESPACE fastspace TO john;

```
postgres=# GRANT CREATE ON TABLESPACE fastspace TO john;
GRANT
```

创建表。

```
CREATE TABLE customer_t1
(
    c_customer_sk          integer,
    c_customer_id          char(5),
    c_first_name           char(6),
    c_last_name            char(8)
);
```

使用简单的SQL语句


```
postgres=#  CREATE TABLE customer_t1
postgres-# (
postgres(#     c_customer_sk            integer,
postgres(#     c_customer_id            char(5),
postgres(#     c_first_name             char(6),
postgres(#     c_last_name              char(8)
postgres(# );
CREATE TABLE
postgres=#
```

向表中插入单行数据。

INSERT INTO customer_t1(c_customer_sk, c_customer_id, c_first_name)
VALUES (3769, 'hello', 'Grace');

```
postgres=#  INSERT INTO customer_t1(c_customer_sk, c_customer_id, c_first_name) VALUES (3769, 'hello', 'Grace');
INSERT 0 1
```

向表中插入多行数据。

```
postgres=# INSERT INTO customer_t1 (c_customer_sk, c_customer_id, c_first_name) VALUES
postgres-#     (6885, 'maps', 'Joes'),
postgres-#     (4321, 'tpcds', 'Lily'),
postgres-# (9527, 'world', 'James');
INSERT 0 3
postgres=#
```

使用简单的SQL语句

更新表中数据。

UPDATE customer_t1 SET c_customer_sk = 9876 WHERE c_customer_sk = 9527;

```
postgres=# UPDATE customer_t1 SET c_customer_sk = 9876 WHERE c_customer_sk = 9527;
UPDATE 1
postgres=#
```

使用系统表pg_tables查询数据库所有表的信息。

SELECT * FROM pg_tables;

```
postgres=# SELECT * FROM pg_tables;
     schemaname     |           tablename           | tableowner | tablespace | hasindexes | hasrules | hastriggers | tablecreator
--------------------+-------------------------------+------------+------------+------------+----------+-------------+-------------
-----
 pg_catalog         | pg_statistic                  | omm        |            | t          | f        | f           |
 public             | customer_t1                   | omm        |            | f          | f        | f           | omm
5+08
 pg_catalog         | pg_type                       | omm        |            | t          | f        | f           |
 pg_catalog         | pg_ts_dict                    | omm        |            | t          | f        | f           |
 pg_catalog         | pg_job_proc                   | omm        | pg_global  | t          | f        | f           |
 pg_catalog         | pg_authid                     | omm        | pg_global  | t          | f        | f           |
 pg_catalog         | pg_statistic_ext              | omm        |            | t          | f        | f           |
 pg_catalog         | gs_wlm_instance_history       | omm        |            | f          | f        | f           |
 pg_catalog         | gs_wlm_session_query_info_all | omm        |            | f          | f        | f           |
 pg_catalog         | gs_wlm_user_resource_history  | omm        |            | f          | f        | f           |
 pg_catalog         | pg_user_mapping               | omm        |            | t          | f        | f           |
```

使用简单的SQL语句

创建schema。

CREATE SCHEMA myschema;

```
postgres=# CREATE SCHEMA myschema;
CREATE SCHEMA
```

在myschema下创建mytable表。

CREATE TABLE myschema.mytable(id int, name varchar(20));

```
postgres=#  CREATE TABLE myschema.mytable(id int, name varchar(20));
CREATE TABLE
```

查询myschema下mytable表的所有数据。

SELECT * FROM myschema.mytable;

```
postgres=# SELECT * FROM myschema.mytable;
 id | name
----+------
(0 rows)
```

使用简单的SQL语句

创建普通视图。

CREATE OR REPLACE VIEW MyView AS SELECT * FROM customer_t1 WHERE c_first_name like ‘G%';

```
postgres=# CREATE OR REPLACE VIEW MyView AS SELECT * FROM customer_t1 WHERE c_first_name like 'G%';
CREATE VIEW
```

查询普通视图。

SELECT * FROM MyView;

```
postgres=#  SELECT * FROM MyView;
 c_customer_sk | c_customer_id | c_first_name | c_last_name
---------------+---------------+--------------+-------------
          3769 | hello         | Grace        |
(1 row)
```

创建索引。

CREATE INDEX cor_all_std_key ON score(cor_id);

```
scg=> CREATE INDEX std_all_cor_key ON score(std_id);
CREATE INDEX
scg=> CREATE INDEX cor_all_std_key ON score(cor_id);
CREATE INDEX
```

使用简单的SQL语句

创建触发器函数。

```
CREATE OR REPLACE FUNCTION func_name() RETURNS TRIGGER AS
$$
DECLARE
BEGIN
        **函数体**
        RETURN NEW/OLD;
END
$$ LANGUAGE PLPGSQL;
```

示例：

```
scg=> CREATE OR REPLACE FUNCTION insert_fail_t_func() RETURNS TRIGGER AS
scg-> $$
scg$> DECLARE
scg$> BEGIN
scg$> IF NEW.grade<60 THEN
scg$> INSERT INTO fail VALUES(NEW.std_id,NEW.cor_id,NEW.grade);
scg$> END IF;
scg$> RETURN NEW;
scg$> END
scg$> $$LANGUAGE PLPGSQL;
CREATE FUNCTION
```

使用简单的SQL语句

创建INSERT触发器。

CREATE TRIGGER trigger_name
AFTER INSERT ON table_name
FOR EACH ROW
EXECUTE PROCEDURE func_name();

创建UPDATE触发器。

CREATE TRIGGER trigger_name
AFTER UPDATE ON table_name
FOR EACH ROW
EXECUTE PROCEDURE func_name();

创建DELETE触发器。

CREATE TRIGGER trigger_name
BEFORE DELETE ON table_name
FOR EACH ROW
EXECUTE PROCEDURE func_name();

示例：

```
scg=> CREATE TRIGGER insert_fail_t
scg-> AFTER INSERT ON score
scg-> FOR EACH ROW
scg-> EXECUTE PROCEDURE insert_fail_t_func();
CREATE TRIGGER
```

使用简单的SQL语句

创建存储过程。

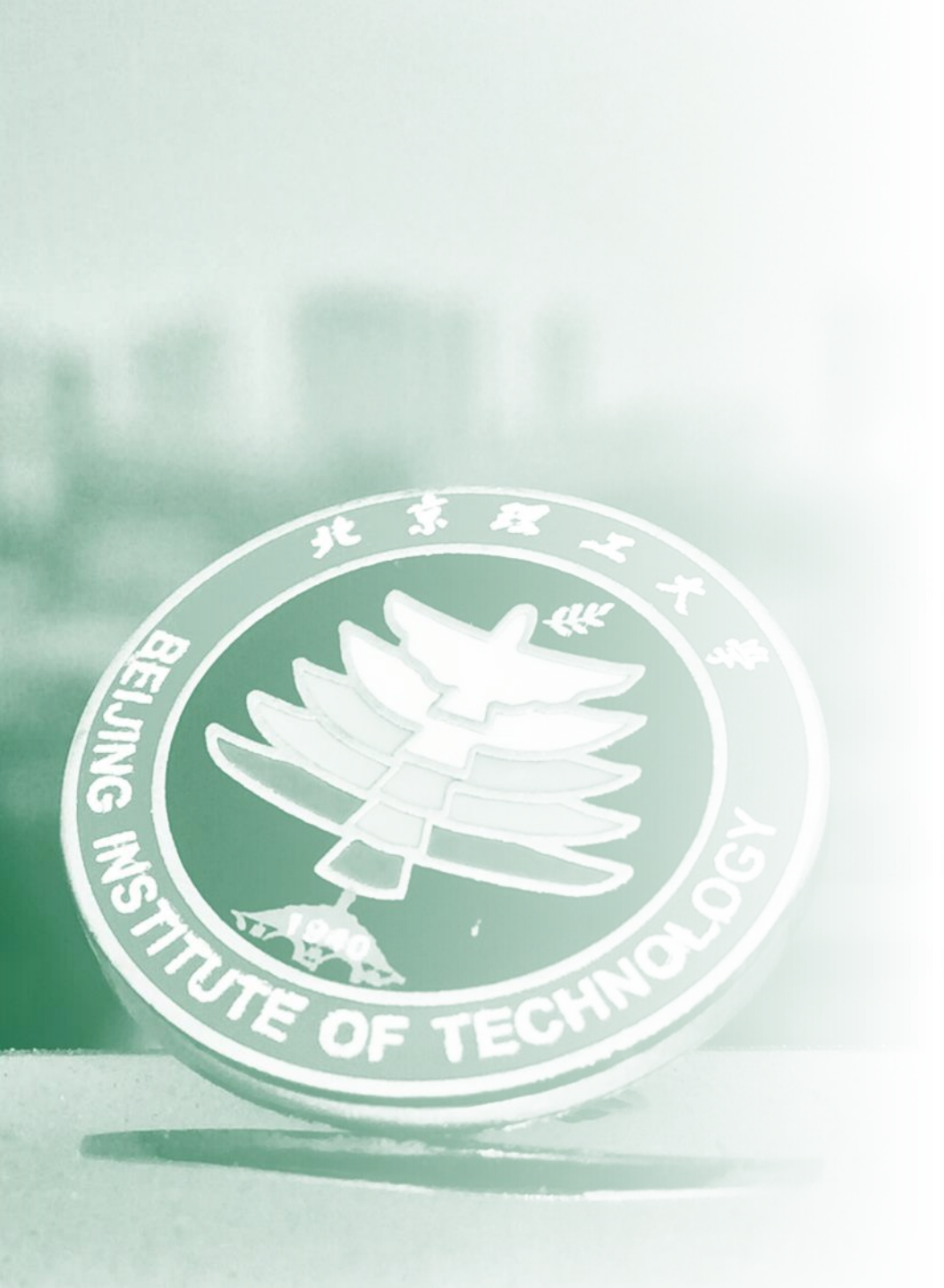
```
create or replace procedure
procedure_name(var_name var_type)
as
declare
        **核心部分要用的变量的声明**
        var_name1 var_type1 := 0;
begin
        **核心部分**
end;
/
```

调用存储过程。

```
call procedure_name(参数值）；
```

示例：

```
postgres=# create or replace procedure cal_gpa(cur_std_id int)
postgres-# as
postgres$# declare
postgres$# credit_sum numeric := 0;
postgres$# gpa_sum numeric := 0;
postgres$# begin
postgres$# if (select count(1) from gpa where std_id = cur_std_id) = 0 then
postgres$# insert into gpa values(cur_std_id, gpa_sum/credit_sum);
postgres$#    else
postgres$#  update gpa set std_gpa = gpa_sum/credit_sum where std_id = cur_std_id;
postgres$#    end if;
postgres$# end;
postgres$# /
CREATE PROCEDURE
```



5 创建并使用IAM帐号

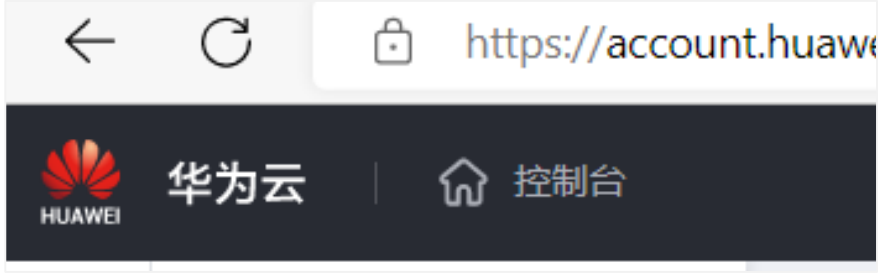
什么是IAM?

IAM是一个身份管理系统，多个IAM用户依附于一个根账号，并且由根账号决定IAM用户的行动能力。

系统默认内置了admin组，用于拥有帐号所有资源的使用及管理权限。 一个组织的管理员需要首先注册并登录华为云。

创建具有不同权限的用户组

登录成功之后，点击左上角“控制台”。

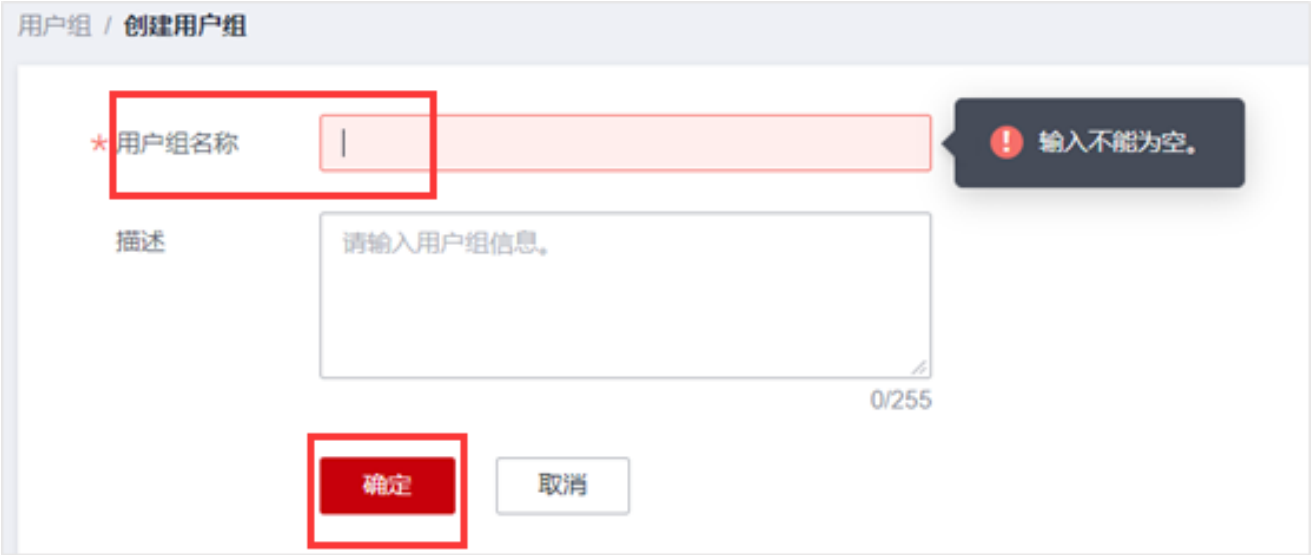


点击右上角帐号的下拉菜单中的“统一身份认证”。

创建具有不同权限的用户组

在统一身份认证服务，左侧导航窗格中，单击“用户组”>“创建用户组”。

在“创建用户组”界面，输入“用户组名称”，单击“确定”，完成用户组创建。



创建具有不同权限的用户组

下面是创建好的用户组（作为例子）。

用户组名称	用户数量	描述	创建时间	操作
测试人员组	0	--	2022/10/16 12:20:20 GMT+08:00	授权 | 编辑 | 用户组管理 | 删除
开发人员组	0	--	2022/10/16 12:20:12 GMT+08:00	授权 | 编辑 | 用户组管理 | 删除
admin	1	拥有所有操作权限的用户组。	2022/09/14 10:37:06 GMT+08:00	授权 | 编辑 | 用户组管理 | 删除

可以在测试人员组的“操作”一栏中，点击“授权”，给测试人员组授权如下。

授权成功

共授予1个权限，信息如下，授权记录可前往【权限管理-授权管理】列表查看

名称	作用范围	类型	描述
APM FullAccess	区域项目	系统策略	应用性能管理服务所有权限

创建具有不同权限的用户组

同样可以给开发人员组授权如下。

授权成功

共授予6个权限，信息如下，授权记录可前往【权限管理-授权管理】列表查看

名称	作用范围	类型	描述
ECS FullAccess	所有资源	系统策略	弹性云服务器所有权限
RDS FullAccess	所有资源	系统策略	关系型数据库服务所有权限
ELB FullAccess	所有资源	系统策略	弹性负载均衡服务所有权限
VPC FullAccess	所有资源	系统策略	虚拟私有云所有权限
EVS FullAccess	所有资源	系统策略	云硬盘的所有权限，具有云硬盘资源的创建，扩容，挂载，卸载，查询，删除等操作权限。
OBS OperateAccess	所有资源	系统策略	具有对象存储服务（OBS）查看桶列表、获取桶元数据、列举桶内对象、查询桶位置、上传对象、

之后该组织就可以使用管理员帐号，给成员创建IAM用户并加入用户组，使得他们拥有独立的用户和密码，可以独立登录华为云平台，并使用他们加入的用户组的权限范围内的资源。

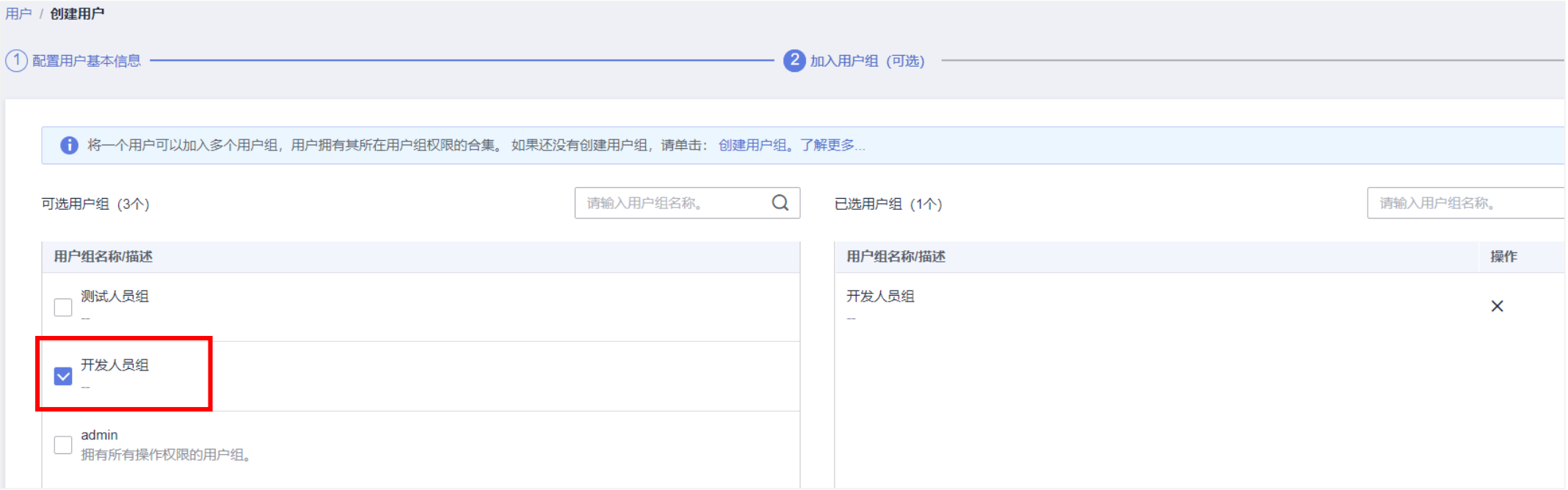
给成员创建IAM用户并加入用户组

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

在管理员的帐号中：在统一身份认证服务，左侧导航窗格中，单击“用户”＞“创建用户”。之后配置用户的基本信息。比如创建用户molly。

给成员创建IAM用户并加入用户组

使molly加入开发人员组。



以同样的流程创建用户mary。以上管理员使用自己的帐号创建了两个IAM用户molly和mary。

给成员创建IAM用户并加入用户组

之后该组织的开发成员就可以使用IAM用户 molly 的身份独立登录华为云平台：首先在登录页面，单击登录下方的“IAM用户”，在“IAM用户登录”页面，输入“租户名/原华为云帐号名”、“IAM用户名”和“密码”，即可登录成功。

北京理工大學
BEIJING INSTITUTE OF TECHNOLOGY

Thank you!